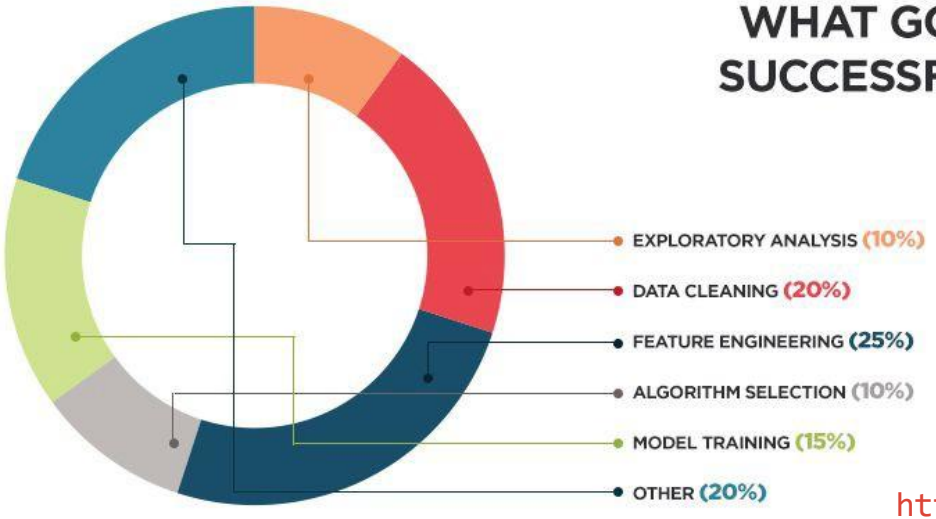
feature engineering

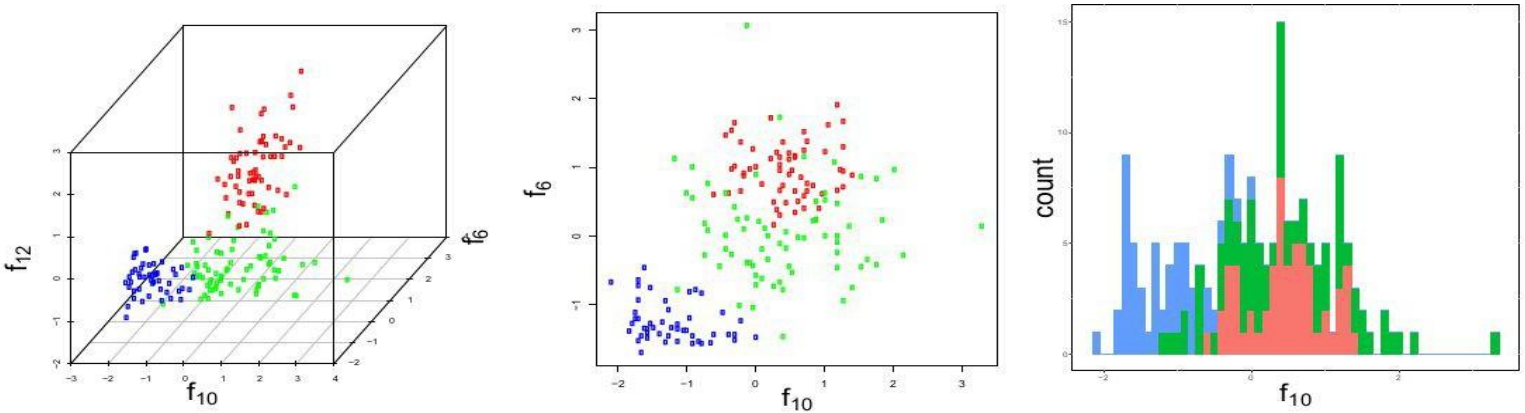
Q: how do you get the most out of your data, for predictive modelling?

Data scientists typically spend a lot of time doing “feature engineering”...



https://elitedatascience.com/feature-engineering

More features = good?



(a) 3 features: f_{10}, f_6, f_{12}. (b) 2 features: f_{10}, f_6. (c) 1 feature: f_{10}.

Figure 1.1: Wine dataset projected across varying numbers of features. Wine contains three classes, 13 features, and 178 instances.

from Andrew Lensen's PhD thesis, VUW, 2019

e.g. plot 3d demo

an imaginary example...

1000 cases, half are ♀ / ♂, 30% have heart disease

frequency of the antecedant (l.h.s), overall

fraction of cases where the consequent (r.h.s.) was true

Table 2 An imaginary example of dependency rules related to heart disease

	Rule	*fr*	ϕ
1	smoking → heart disease	120	0.400
2	sports → ¬ heart disease	400	0.800
3	coffee → ¬ heart disease	240	0.700
4	stress → heart disease	150	0.300
5	pine bark extract → ¬ heart disease	1	1.000
6	female → ¬ heart disease	352	0.704
7	female, stress → heart disease	100	0.385
8	stress, smoking → heart disease	100	0.500
9	smoking, coffee → heart disease	96	0.400
10	smoking, sports → heart disease	20	0.333
11	female, sports → ¬ heart disease	203	0.808

simple dependencies

independencies

spurious (too few)

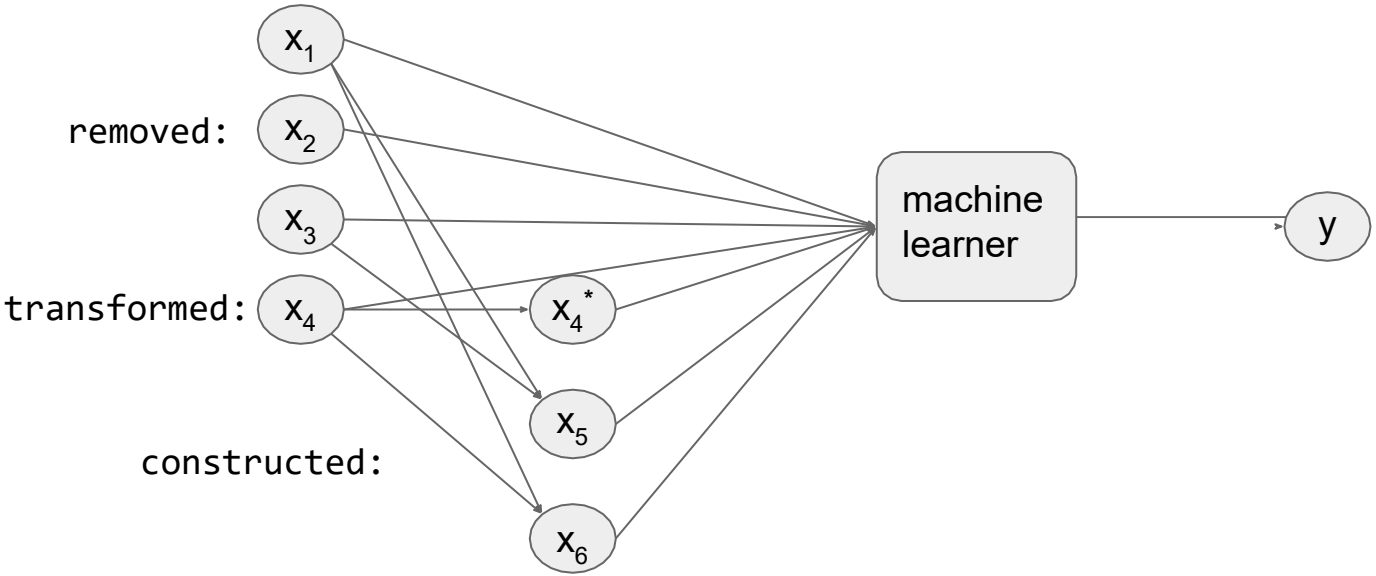
spurious (too *small*)

non-monotonicity!

superfluous variables

http://i.giwebb.com/2019/02/25/tutorial-on-statistically-sound-pattern-discovery/

changes to features

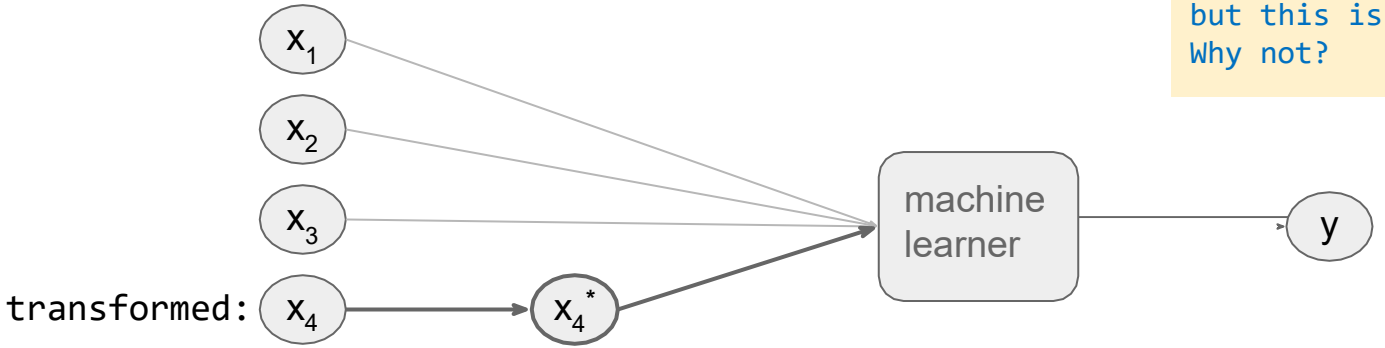


one-to-one

aside: if $x_1 \in$ {dog, cat, ferret} it is tempting to transform into

dog → 1
cat → 2
ferret → 3

but this is *not* a good idea.
Why not?

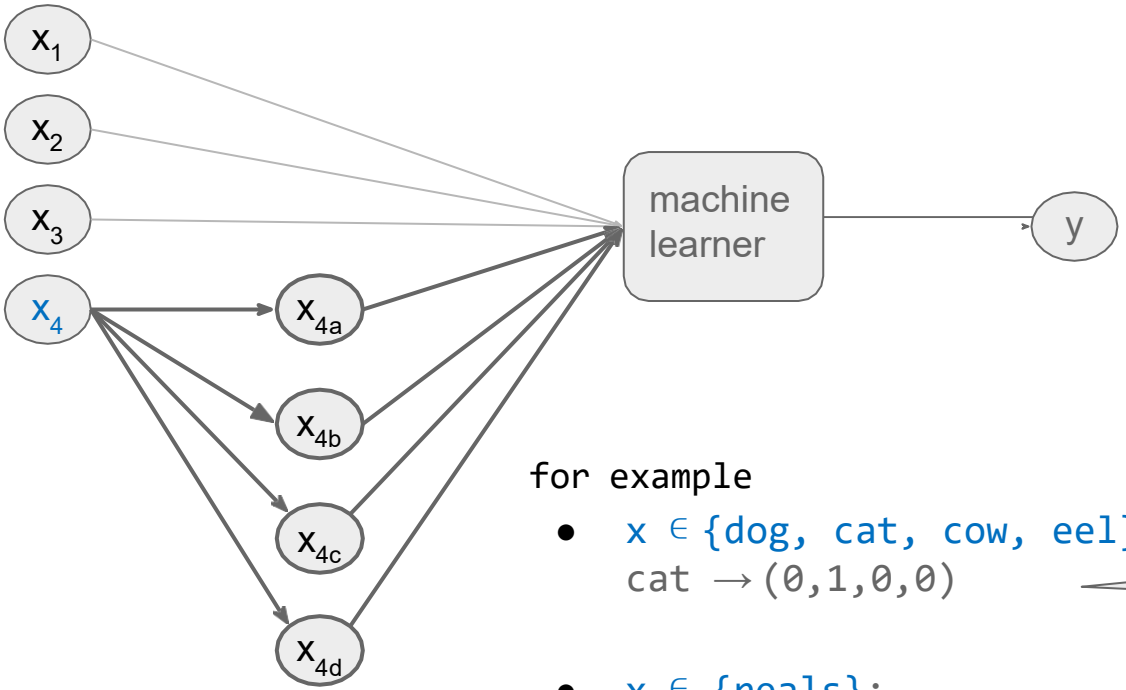


can get the original feature into a more useful range for the learner

for example

- clipping
- normalization
- discretization
- mathematical transformations such as logarithm log(x), reciprocal function 1/x, exponentiation exp(x), etc…

one-to-many



might make it easier / more natural for the machine learner to work with

for example

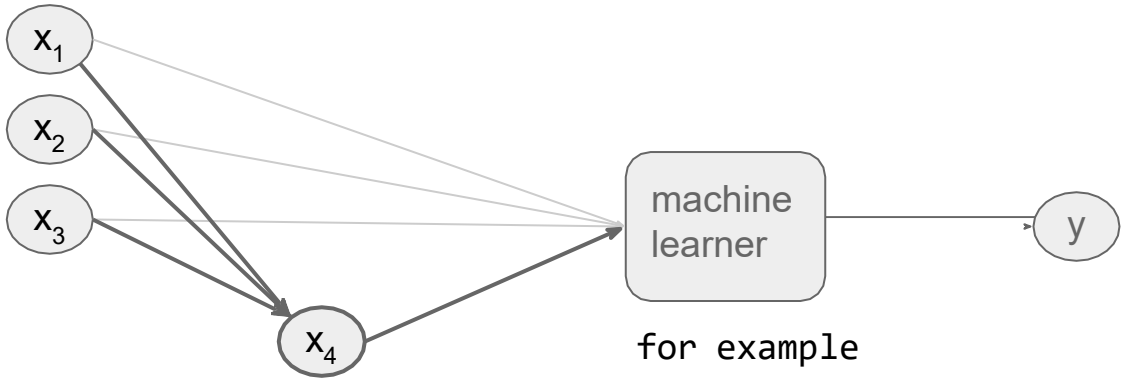
- $x \in \{dog, cat, cow, eel\}$:
  $cat \rightarrow (0,1,0,0)$

  "one-hot" encoding

- $x \in \{reals\}$:
  $x \rightarrow (x, x^2, x^3, x^4)$

  polynomial expansion

many-to-one

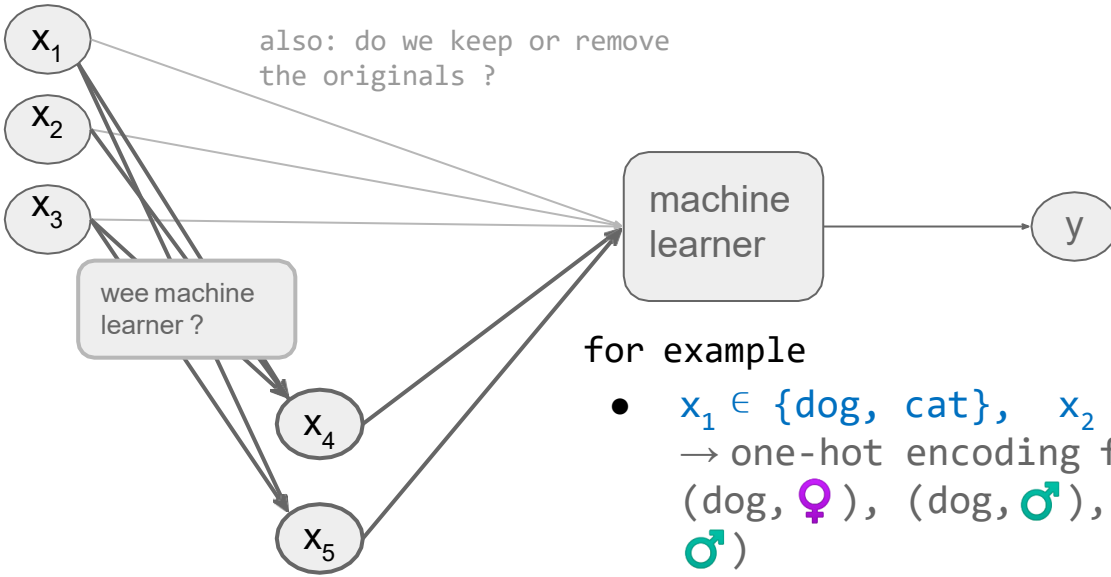


put basic features together to construct new features that can capture relationships between the basic features – the idea is to augment the feature space with these too

for example

- $x \in$ {TRUE, FALSE}:
  $(x_1$ and $x_2)$ or x_3
- $x \in$ {reals}:
  $(x_1 + x_2) * x_3$

many-to-many



known as the cartesian product; allows learner to be "intersectional"

for example

- $x_1 \in$ {dog, cat}, $x_2 \in$ {♀, ♂} :
  → one-hot encoding from (dog, ♀), (dog, ♂), (cat, ♀), (cat, ♂)

can consolidate redundant information across features

- $x \in$ {reals}:
  → PCA? kernel PCA?
  → ICA?
  → a one-hot encoding from clustering?

many-to-one is a.k.a. “feature construction”

- we can also put basic features together to make constructed features that might capture relationships between the basic features – the idea is to augment the feature space with these too

E.g. Given base features [X_1, X_2, X_3]

linear constructions: $Xc = X_1+X_2$,

or $Xc = 4X_1+3X_2+6X_3$,

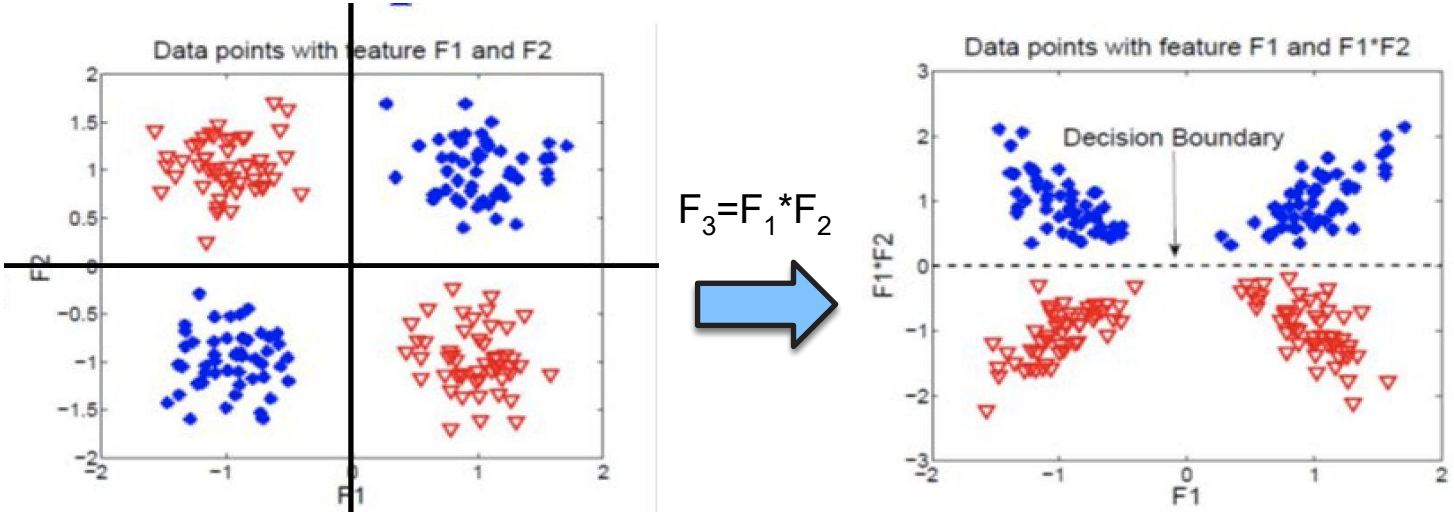
…

nonlinear constructions: $Xc = X_1*X_2$,

feature construction

the quality of such constructed features can drastically affect the learning performance

- -ve: Feature interactions introduce errors and add complexity
- +ve: more meaningful features that lead to more concise and accurate machine learning models



components of a feature construction system

Automatic feature construction systems need the following two components:

- a search strategy to search the space of possible functions (of original features)
- an evaluation mechanism to measure the goodness of a candidate function

operators for constructing new features

Boolean features:

- x1, x2 ∈ {TRUE, FALSE}
  - e.g. x3 ← not(x1)
  - e.g. x4 ← xor(x1,x2)

⇒ logical operators like negation, conjunctions, disjunctions,...

Nominal/Categorical features:

- x1 ∈ {dog, cat},x2 ∈ {rain, snow}...
  - e.g. x3 ∈ {(dog,rain), (dog,snow), (cat,rain), (cat,snow)}

⇒ cartesian product

Numerical features:

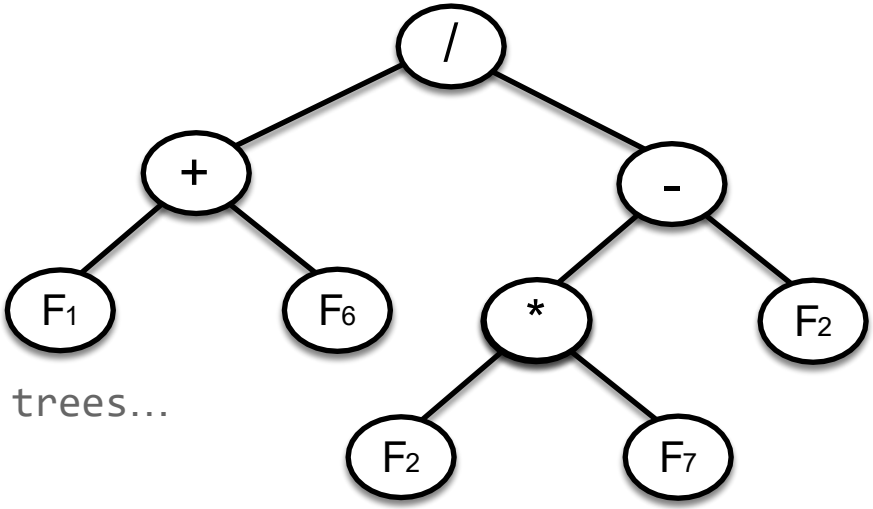
- x1 ∈ {ints}, x2 ∈ {reals},...
  - x4 ← min(x1, mean(x2,x3))

⇒ add, multiply, max, average, equals, ...

can we construct new features automatically with Genetic Programming?

Genetic Programming is a flexible way to make mathematical and logical functions

warning: there isn't much structural (topological) information in the search space of possible functions



→ yes it's possible,
but be prepared to spend a lot of compute time trying out dud trees…

evaluating new features

- Not all constructed features are good!...

- Apply the usual feature selection techniques to remove redundant and irrelevant features
    - Q: what are the “usual” techniques?

an imaginary example...

1000 cases, half are men/women, 30% have heart disease

frequency of the antecedant (l.h.s), overall

fraction of cases where the consequent (r.h.s.) was true

	Rule	*fr*	ϕ
1	smoking → heart disease	120	0.400
2	sports → ¬ heart disease	400	0.800
3	coffee → ¬ heart disease	240	0.700
4	stress → heart disease	150	0.300
5	pine bark extract → ¬ heart disease	1	1.000
6	female → ¬ heart disease	352	0.704
7	female, stress → heart disease	100	0.385
8	stress, smoking → heart disease	100	0.500
9	smoking, coffee → heart disease	96	0.400
10	smoking, sports → heart disease	20	0.333
11	female, sports → ¬ heart disease	203	0.808

simple dependencies

independencies

spurious (too few)

spurious (too *small*)

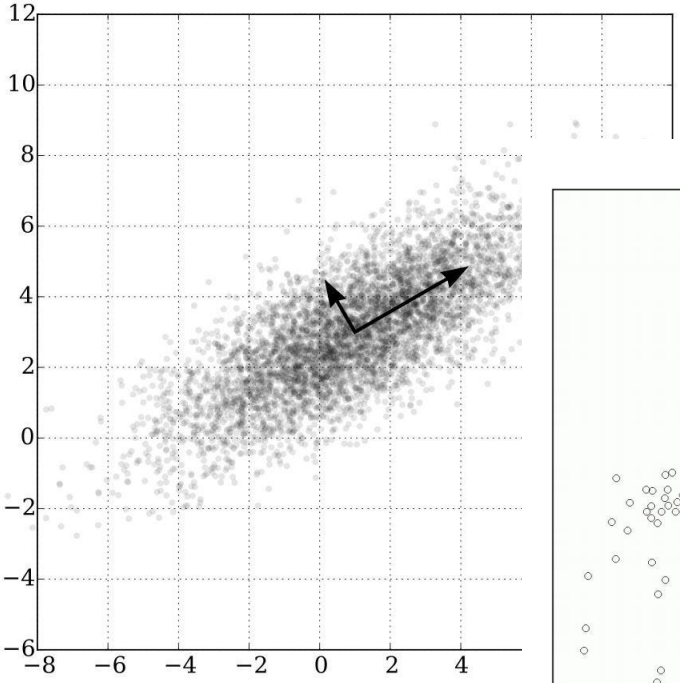
non-monotonicity!

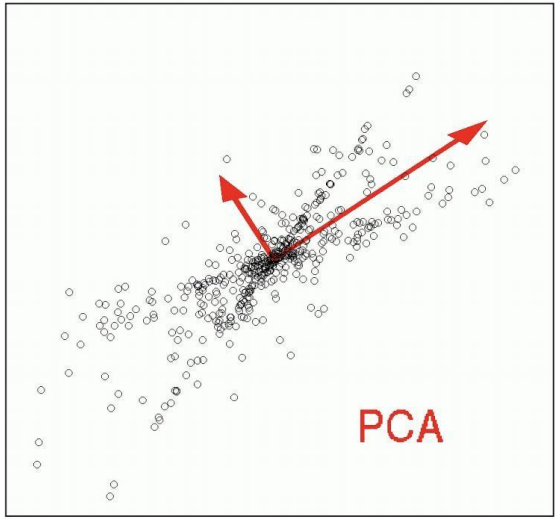
superfluous variables

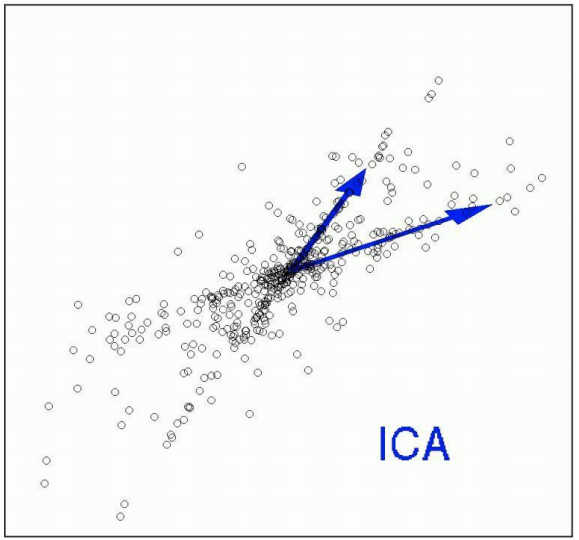
Some specific “many to many” feature methods:

- PCA
- ICA
- Kernel PCA

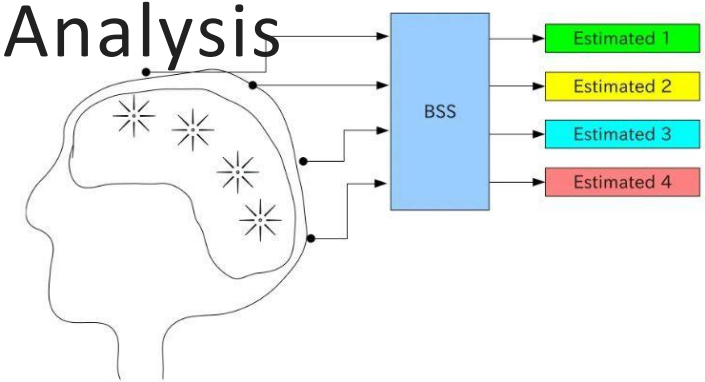
PCA (Principal Components Analysis)







ICA - Independent Components Analysis



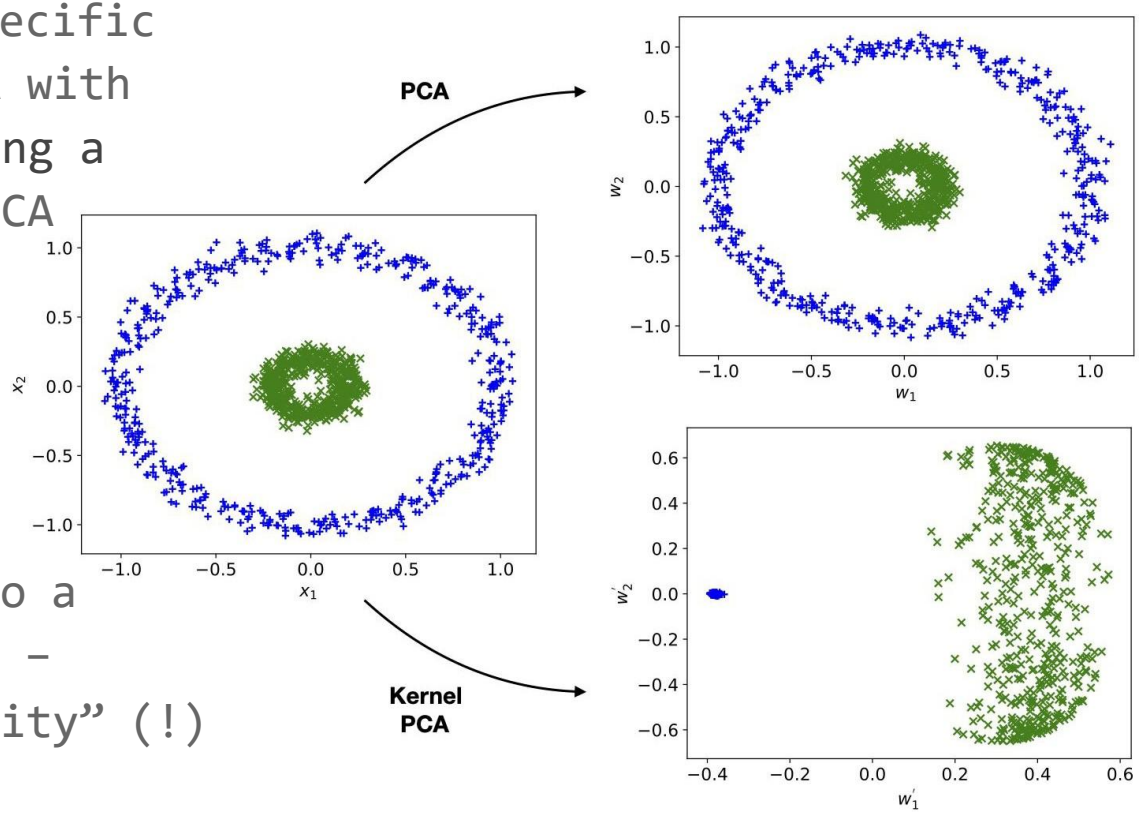
- PCA and ICA both create new components that are linear combinations of the originals

- ICA assumes the observed data arose as a linear combination of independent components (that are sadly unknown)
  - Goal of ICA: recover the components from X by finding a linear transformation
- it's like PCA but we're looking for components that are as statistically independent as possible
- PCA removes correlation, but doesn't usually find those independent components that might actually underly the data
- why might it be good to find truly independent components?

kernel PCA

- Kernel PCA combines a specific mathematical view of PCA with kernel functions, creating a nonlinear extension of PCA

- Perform PCA on data that has been transformed into a high-dimensional setting – “blessing of dimensionality” (!)



https://ml-lectures.org/docs/structuring_data/ml_without_neural_network-2.html

missing values

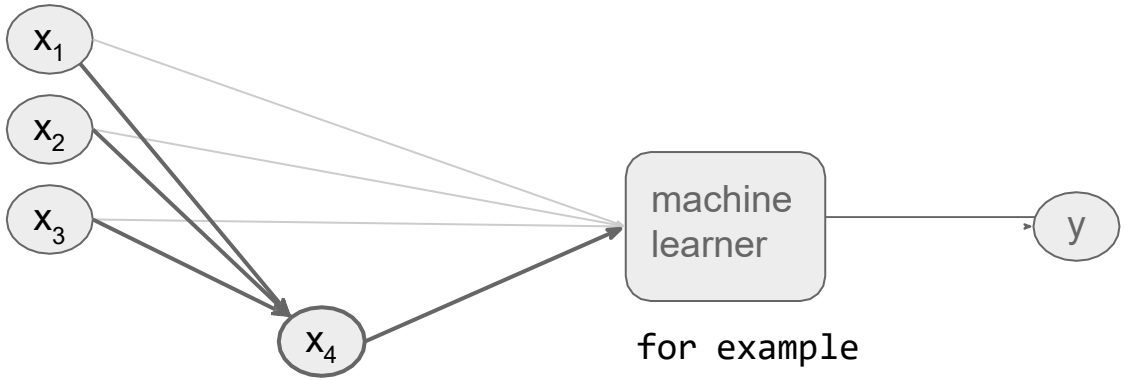
but first: finishing off previous lecture



L1: feature engineering

L2: imputing missing data

many-to-one



put basic features together to construct new features that can capture relationships between the basic features – the idea is to augment the feature space with these too

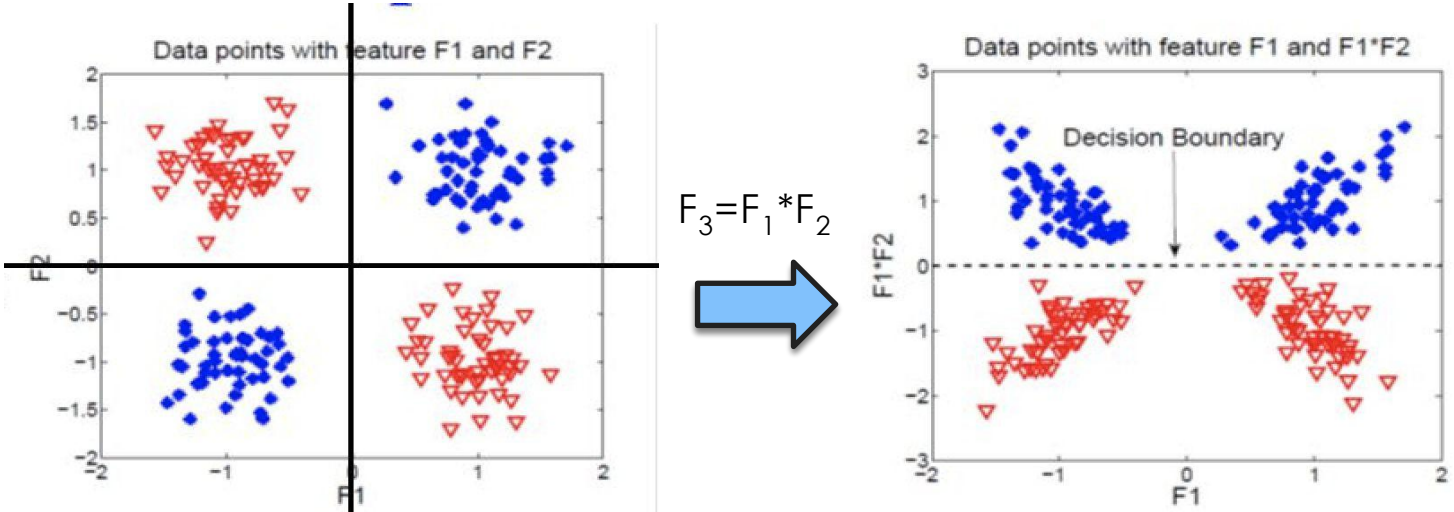
for example

- $x \in$ {TRUE, FALSE}:
  $(x_1$ and $x_2)$ or x_3
- $x \in$ {reals}:
  $(x_1 + x_2) * x_3$

feature construction (a.k.a. “many-to-one”)

the quality of such constructed features can drastically affect the learning performance

- -ve: Feature interactions introduce errors and add complexity
- +ve: more meaningful features that lead to more concise and accurate machine learning models



components of a feature construction system

Automatic feature construction systems need the following two components:

- a search strategy to search the space of possible functions (of original features)
- an evaluation mechanism to measure the goodness of a candidate function

operators for constructing new features

Boolean features:

- x1, x2 ∈ {TRUE, FALSE}
  - e.g. x3 ← not(x1)
  - e.g. x4 ← xor(x1,x2)

⇒ logical operators like negation, conjunctions, disjunctions,...

Nominal/Categorical features:

- x1 ∈ {dog, cat},x2 ∈ {rain, snow}...
  - e.g. x3 ∈ {(dog,rain), (dog,snow), (cat,rain), (cat,snow)}

⇒ cartesian product

Numerical features:

- x1 ∈ {ints}, x2 ∈ {reals},...
  - x4 ← min(x1, mean(x2,x3))

⇒ add, multiply, max, average, equals, ...

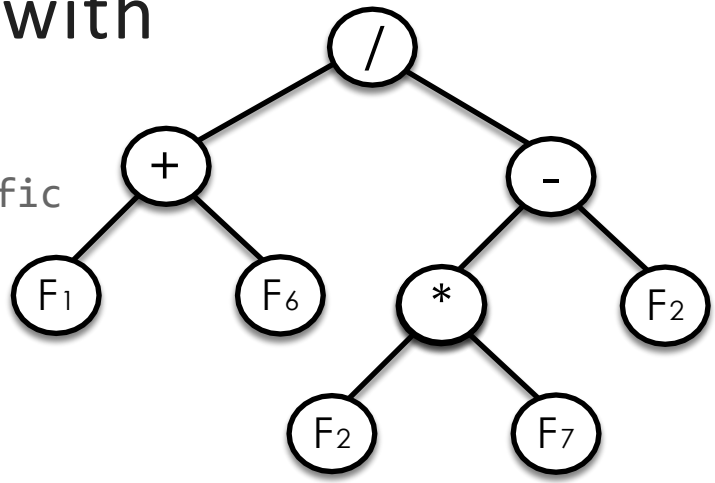
Genetic Programming is a flexible way to make mathematical and logical functions

but warning: there isn't much structural (topological) information in the search space of possible functions

→ yes it's possible,
but be prepared to spend a lot of compute time trying out dud trees…

→ Also: just evolving good answers to specific problems is not a recipe for insight. Insight comes from having theories about the world (a.k.a. "a model") and testing it.

construct new features automatically with Genetic Programming?



Evaluating new features

- Not all constructed features are useful or good!...

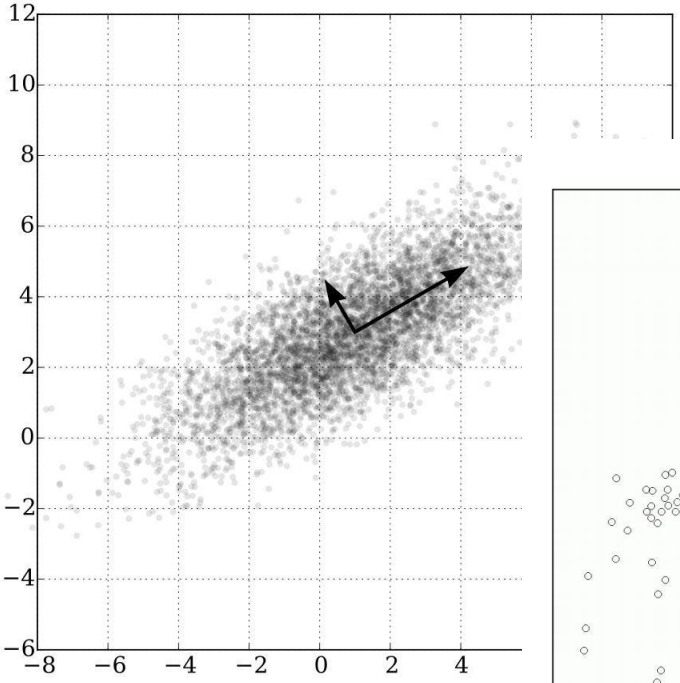
- Apply the usual feature selection techniques to remove redundant and irrelevant features

comment: it is *not necessarily the case* that “redundant” features should be culled.

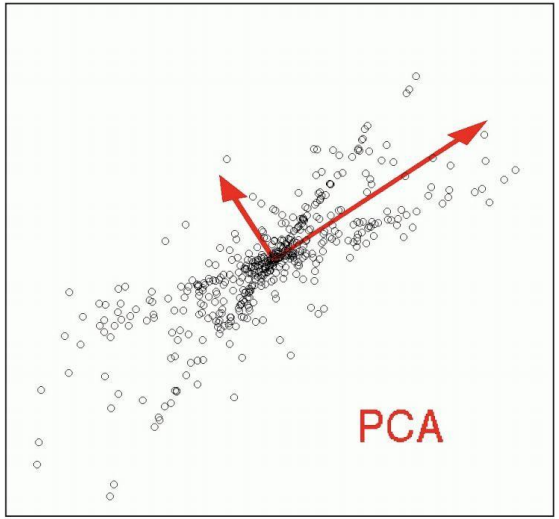
Imagine having three noisy sensors that all measure the same quantity. You would be better off keeping all 3 (to average them to fight the noise…) than to discard two as being “redundant”

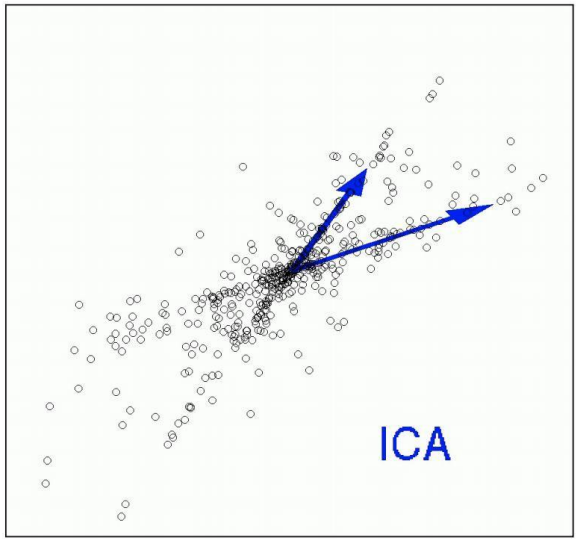
Some specific “many to many” feature methods:

PCA (Principal Components Analysis)



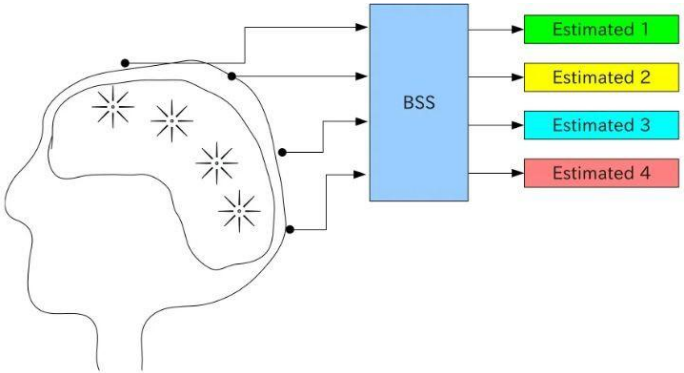
- PCA
- ICA
- Kernel PCA
- autoencocers





ICA = Independent Components Analysis

(a.k.a. “Blind Source Separation”, solving the “cocktail party problem”)

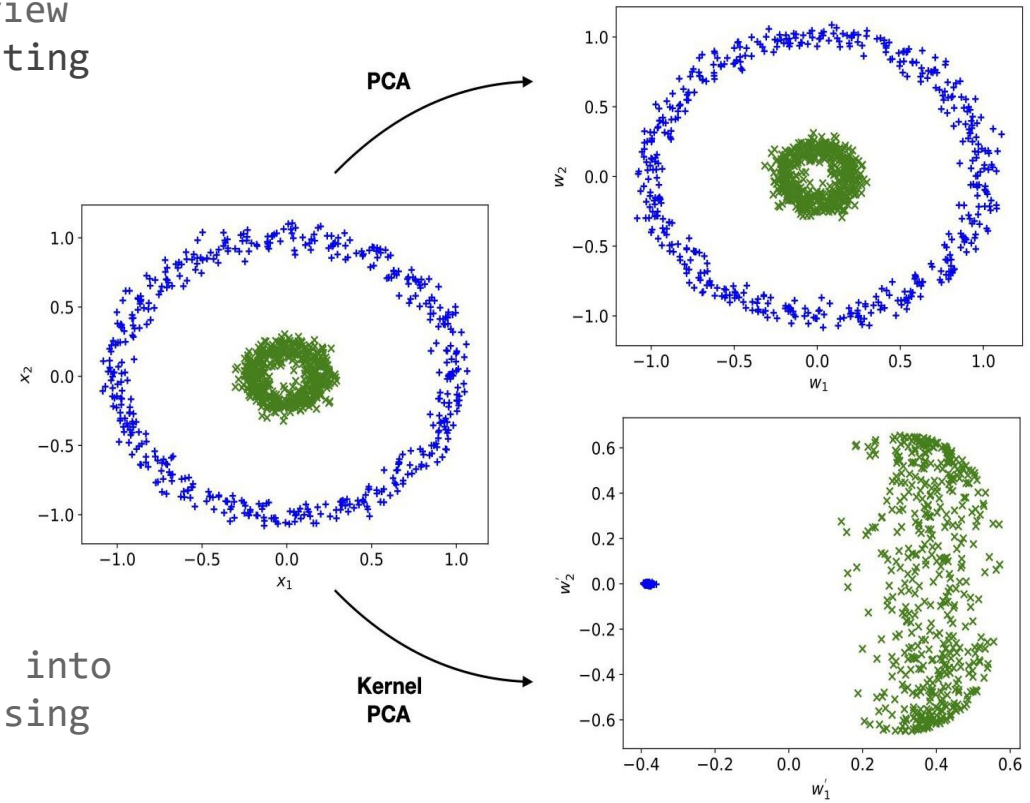


- PCA and ICA both create new components that are linear combinations of the originals
- PCA removes correlation, but doesn’t usually find those independent components that might actually underly the data
- ICA assumes the observed data arose as a linear combination of some independent components (that are tragically unknown). ICA then aims to *recover* those components from data set X.
- ICA finds components that are as statistically independent as possible

Q: why might it be good to find truly *independent* components?

kernel PCA

- combines a specific mathematical view of PCA with kernel functions, creating a nonlinear extension of PCA

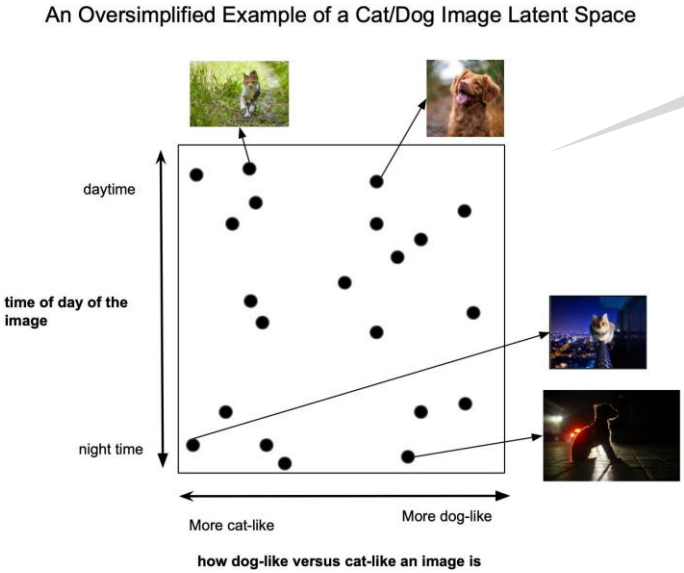


- Perform PCA on data that has (effectively) been transformed into a high-dimensional setting – “blessing of dimensionality” (!)

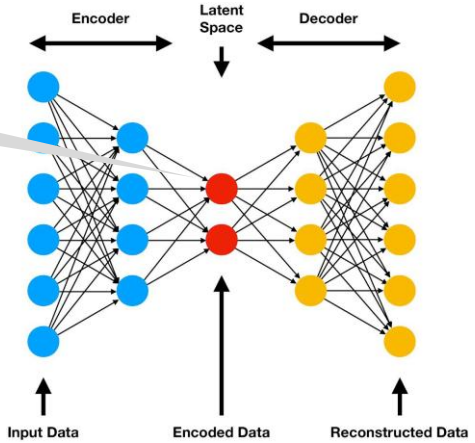
images from https://ml.berkeley.edu/blog/posts/vq-vae/

auto-encoders

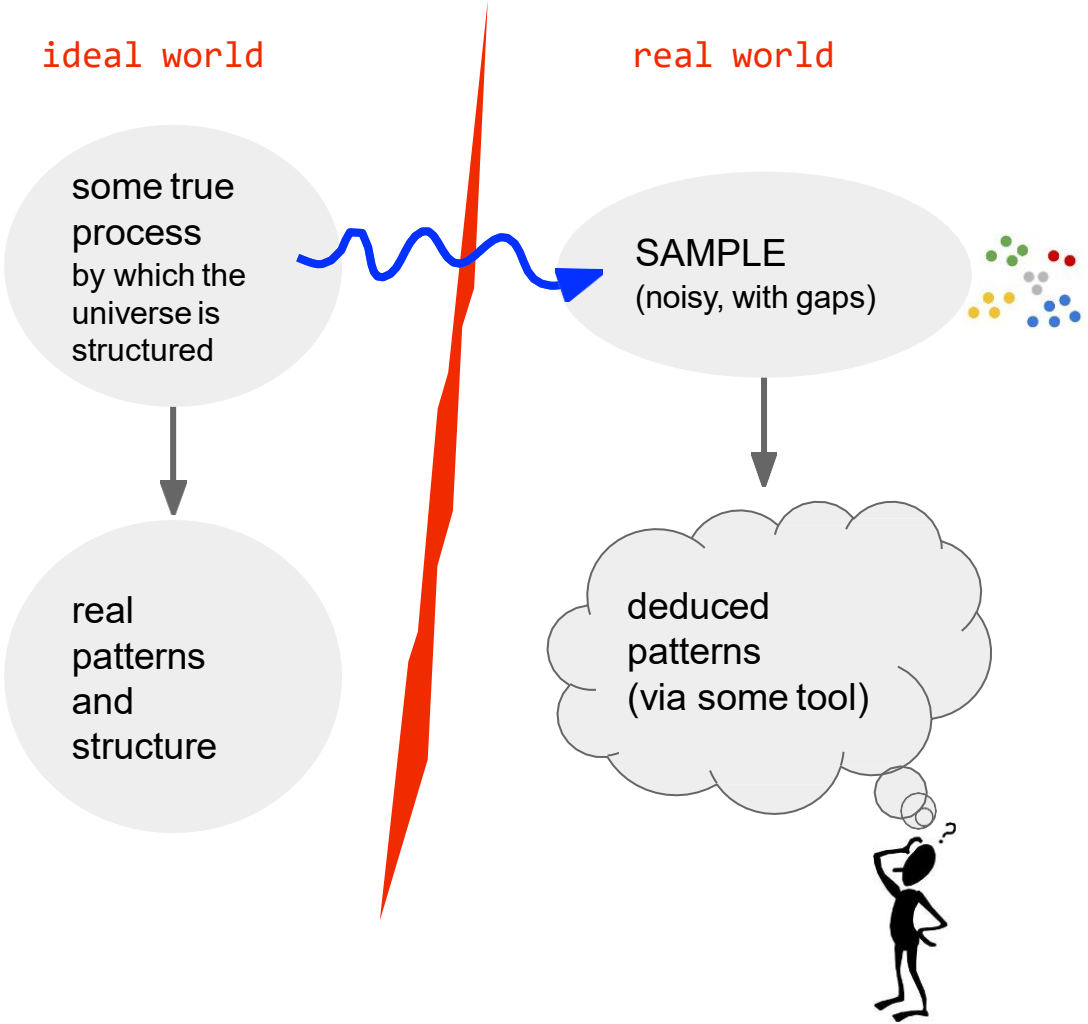
auto-encoders are a type of neural network, trained to “squash” data through a bottleneck layer with just a few neurons – i.e. they do non-linear dimensionality reduction



often referred to as a “latent space”



the problem of finding true patterns from sample data



Missing values

many possible reasons, e.g:

➔ high cost involved in measuring variables,
➔ failure of sensors,
➔ reluctance of respondents to answer some q,
➔ an ill-designed survey

1. Real-world datasets often contain many missing values!
2. Most machine learning algorithms *require* all the values, for a given training example.



You can see the problem!

4 solutions:

- Remove the missing data instances from the dataset, or
- Remove the worst offending attributes from the dataset, or
- Use imputation methods to substitute in “fake” values, or
- Use a learner that is able to cope with missing values

Truly

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

ideal world



real world

Recorded

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	?	No
2	?	?	100K	No
3	No	Single	70K	No
4	Yes	Married	?	No
5	No	?	95K	Yes
6	?	Married	60K	No
7	Yes	Divorced	220K	No
8	No	?	?	Yes
9	?	Married	75K	No
10	No	Single	90K	Yes

- Missing data can also be in the form of features, or labels (just one class, or…?) or both

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	?	?	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	?	95K	Yes
6	?	Married	60K	No
7	Yes	Divorced	220K	No
8	No	?	85K	Yes
9	?	Married	75K	No
10	No	Single	90K	Yes

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	?
2	No	Married	100K	?
3	No	Single	70K	No
4	Yes	Married	120K	?
5	No	Divorced	95K	Yes
6	No	Married	60K	?
7	Yes	Divorced	220K	No
8	No	Single	85K	?
9	No	Married	75K	No
10	No	Single	90K	Yes

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	?	No
2	?	?	100K	?
3	No	Single	?	No
4	Yes	Married	120K	?
5	No	?	95K	Yes
6	?	Married	60K	?
7	Yes	Divorced	?	No
8	No	Single	85K	?
9	No	?	75K	No
10	?	Single	?	Yes

Option 1: just eliminate entire observations

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	?	Married	?	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	?	Divorced	?	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	?	?	75K	No
10	No	Single	90K	Yes



Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
10	No	Single	90K	Yes

This seems extreme! It wastes lots of precious, expensive, data. But let's just say we do this. **Is it even "okay" ?**

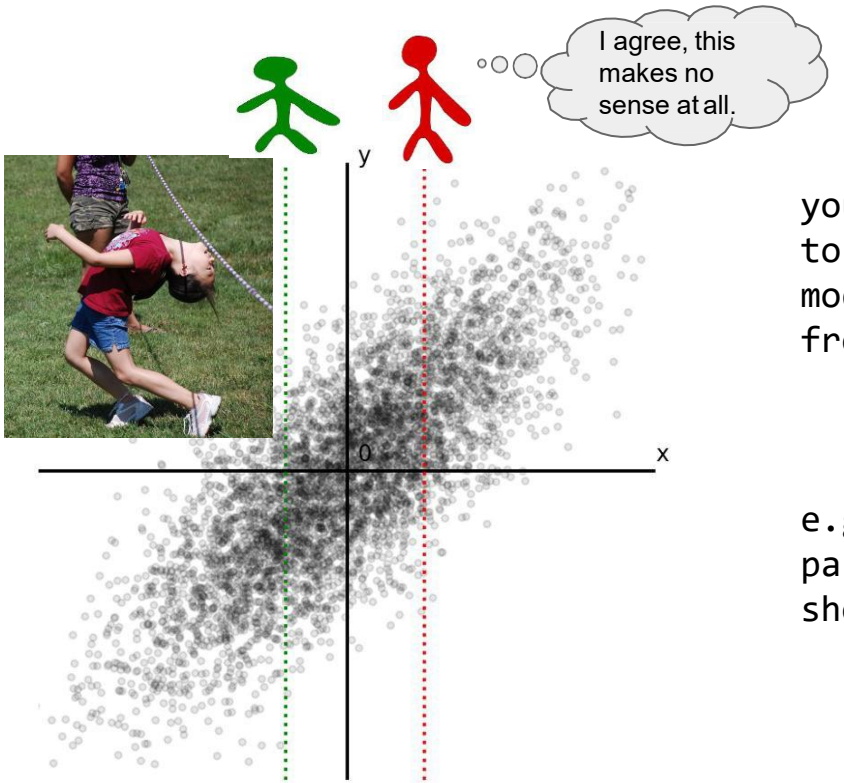
(e.g. suppose we knew *high values of* income were more likely to be missing…)

there are different kinds of missing!

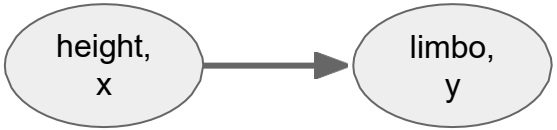
- MCAR missing, completely at random
- MAR missing at random
- MNAR missing, not at random

a toy example

x is height, y is ability to limbo (relative to their average values)

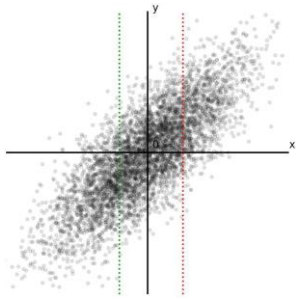


you're using this data to train a regression model to predict y, from x

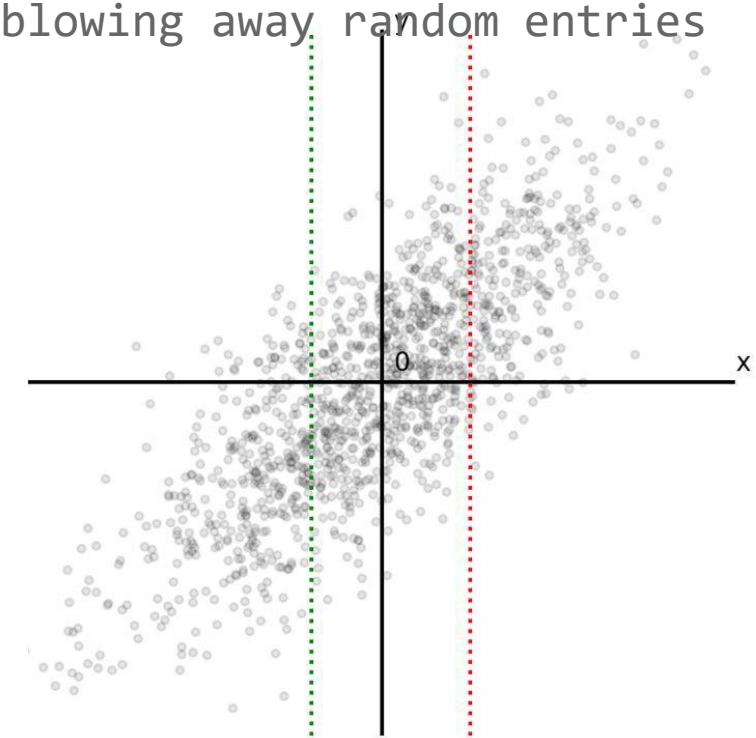


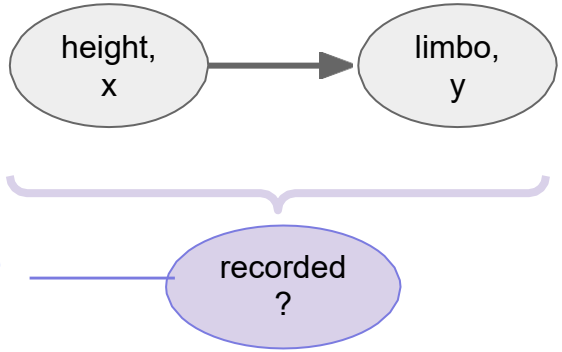
e.g. consider two particular cases, shown as ..... and .....

Missing Completely At Random (MCAR)



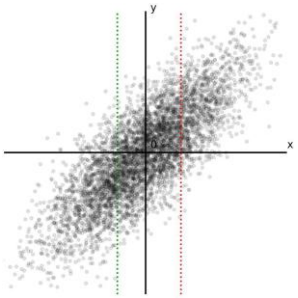
E.g.
I blasted my spreadsheet with a shotgun,
blowing away random entries



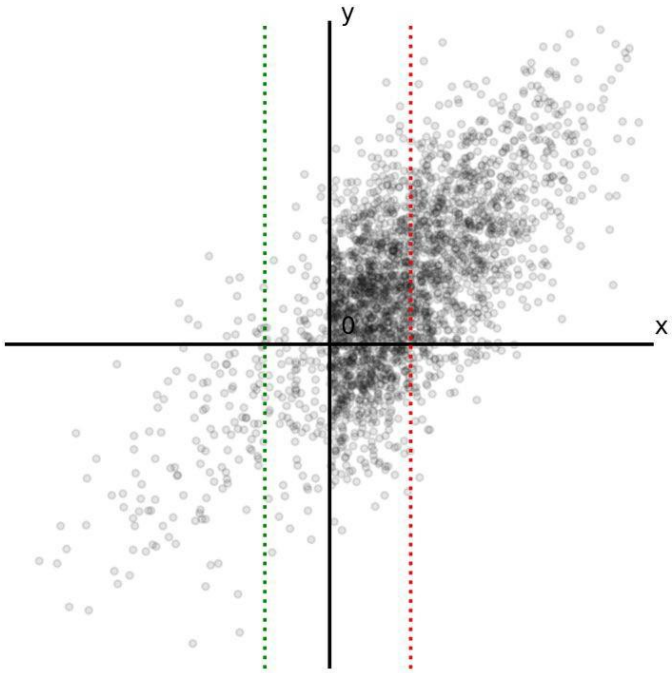


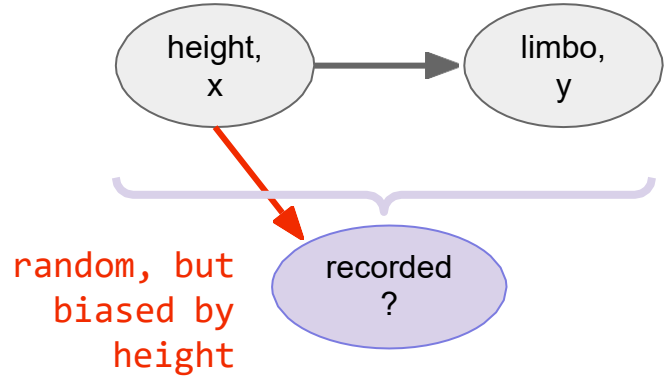
random, like
a coin toss

Missing At Random (MAR)

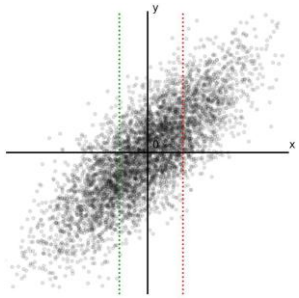


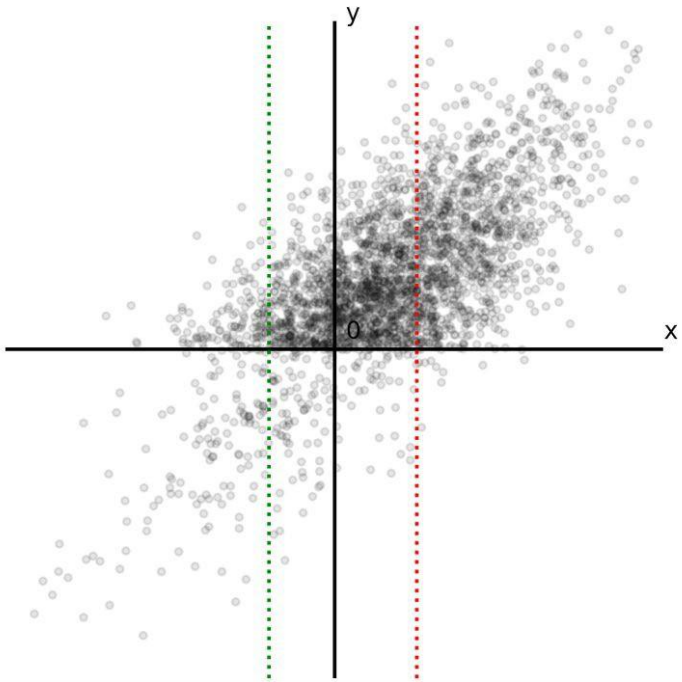
note this name is really confusing. A better name for MAR would have been be “Missing, conditionally at random”, but that would screw with the acronyms 🙄

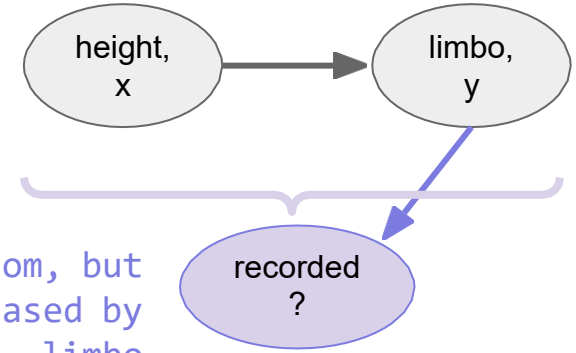




Missing Not At Random (MNAR)



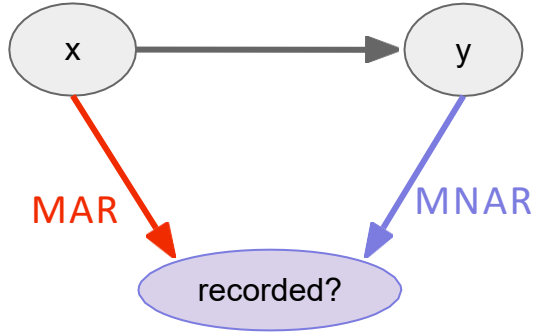




random, but biased by limbo

e.g.

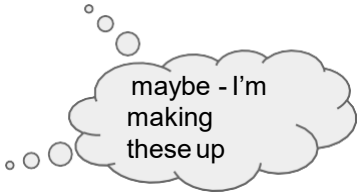
MCAR
has no linkage



1. Half the survey forms weren't delivered or got lost, due to random beetle infestations

2. Males are less likely to fill in a depression survey, but this has nothing to do with their level of depression, after accounting for maleness

3. Depressed people are less likely to fill in a survey about depression
   (similarly, happy people are less likely to fill in a survey about a grievance)



Q. would you say these are cases of MCAR, MAR, or MNAR?

…. tuesday’s lecture got to this point…..

conclusions?

So: simply avoiding data that has missing values isn't ideal

- reason 1?
  - wastes valuable data - what if *every* row had something missing?!
- reason 2?
  - could be MNAR, in which case results will be "biased" (ie. wrong)

but what else can we do?

We could "impute" data – replace missing with actual values.
But what values?!

Option 2: eliminate entire attributes

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	?	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	?	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	?	Married	75K	No
10	No	Single	90K	Yes



Tid		Marital Status	Taxable Income	Cheat
1		Single	125K	No
2		Married	100K	No
3		Single	70K	No
4		Married	120K	No
5		Divorced	95K	Yes
6		Married	60K	No
7		Divorced	220K	No
8		Single	85K	Yes
9		Married	75K	No
10		Single	90K	Yes

Seems a bit drastic

Option 3: Imputation

- Mean imputation
- Mode imputation
- Hot deck imputation
- other ideas…
- Multiple imputation
- MICE (used in sklearn)

impute with Mean, or Mode

- Mean: for continuous attributes
  - Fill in with average complete values

- Mode: for categorical attributes
  - Fill in with the most frequent value

Pros and cons:

(+) Doesn't changes the mean/mode of attributes

(-) Under-represents the variability in the data

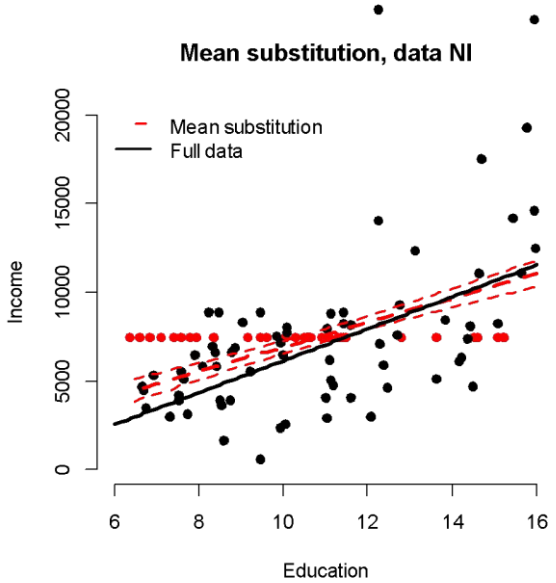
pretty bad imputation: Mean or Mode

Tid	Refund	Marital Status	Taxable Income	Cheat
1	?	Single	125K	No
2	No	?	?	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	?	Yes
6	No	?	60K	No
7	Yes	Divorced	220K	No
8	?	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes
	No	**Single**	**105K**	

most common/ mean value

Tid	Refund	Marital Status	Taxable Income	Cheat
1	**No**	Single	125K	No
2	No	**Single**	**105K**	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	**105K**	Yes
6	No	**Single**	60K	No
7	Yes	Divorced	220K	No
8	**No**	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

sklearn's SimpleImputer has options



“hot deck” imputation

For each record containing missing values:
Find the most similar record, then fill missing values with the corresponding values from that!

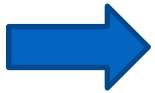
Pros and cons

(+) Imputes realistic values

(-) what do you think?

Imputation by nearest neighbor?

Tid	Refund	Marital Status	Taxable Income	Cheat
1	?	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	?	Divorced	95K	Yes
6	?	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

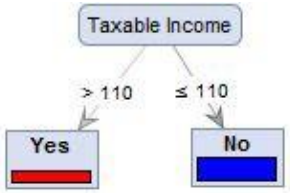


Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

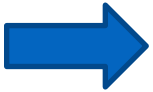


Tid	Refund	Marital Status	Taxable Income	Cheat
1	No	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Imputation using a decision rule?



Tid	Refund	Marital Status	Taxable Income	Cheat
1	?	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	?	Divorced	95K	Yes
6	?	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



Tid	Refund	Marital Status	Taxable Income	Cheat
1	**Yes**	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	**No**	Divorced	95K	Yes
6	**No**	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Imputation using Regression?

- For each attribute containing missing values:
  - Divide data into two parts: the records that are complete, the records having missing values
  - Use the complete data to estimate a regression model between the attribute and others, and use that to predict/suggest missing values in the attribute

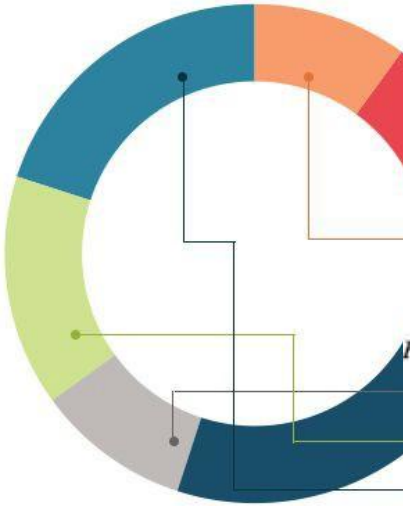
Pros and cons

(+) Maintains relationships between attributes

(-) Makes an assumption about regression model

Multiple Imputation

idea: to avoid bias caused by one imputation method, do several!



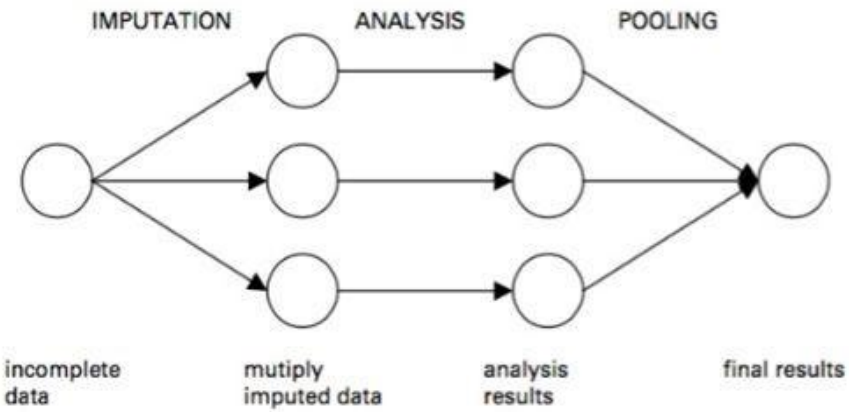
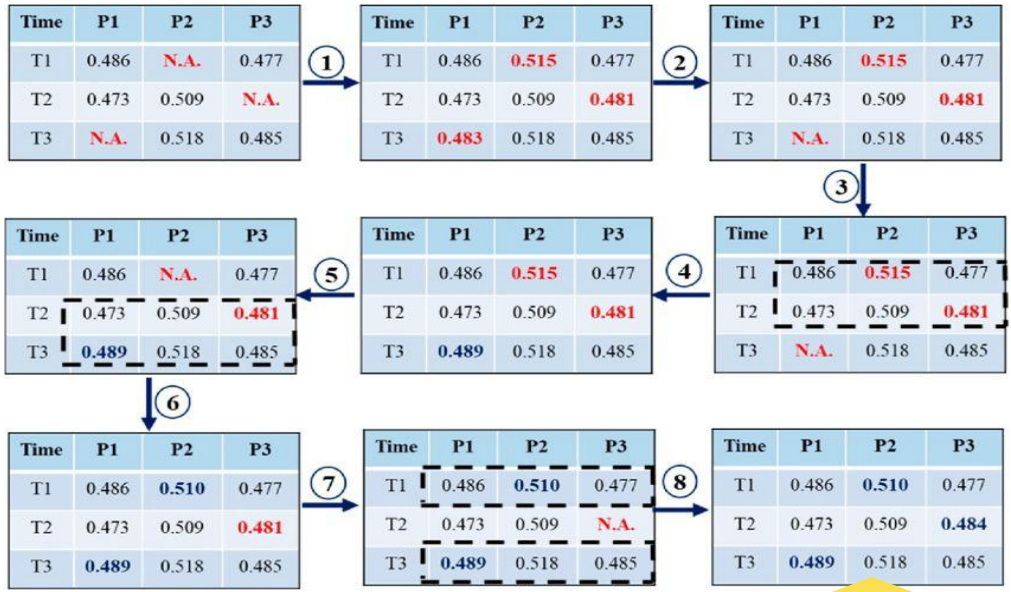


Figure : *Schematic representation of the steps in multiple imputation. The process starts with an incomplete data set (on the left side), which is imputed m times (m=3 here) thus creating m completed data sets. Each complete data set is analyzed by using standard complete-data software, thus resulting in m analysis results. Finally, these m results are pooled into one final result that adequately reflects the amount of uncertainty in the estimates.*

MICE ("Multivariate Imputation by Chained Equations" 😕)

a version of multiple imputation that apparently works quite well and is implemented in sklearn:

1. Fill all missing values with random values (?!?)
2. Regress each attribute that contains missing values on other attributes, in a chain => 1 imputed dataset



sklearn's IterativeImputer does this, which is steps 1 and 2

3. Repeat whole procedure N times to generate N imputed datasets
4. Average N imputed datasets => final imputed dataset

https://www.researchgate.net/figure/Workflow-of-multiple-imputation-by-chained-equations-MICE_fig5_350620190

Option 4: use a learning algorithm that can cope with missing values

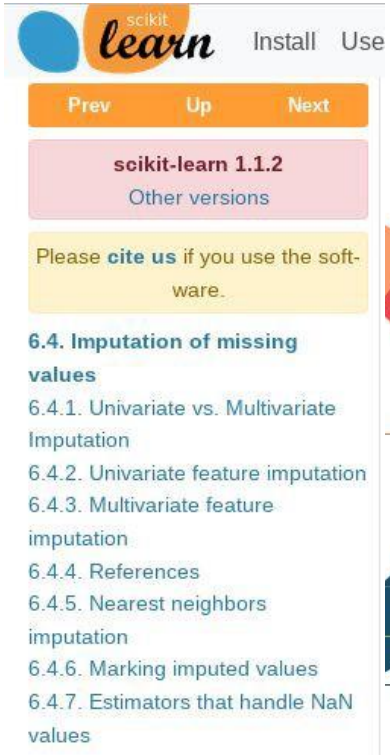
- Missing values can sometimes be taken into account during the learning process of acquiring knowledge

- For example:
  - Clustering algorithms: similarity between the objects can calculated using only the attributes that do not have missing values.
  - Classification: Gradient Boosting, kNN, and Random Forests



scikit-learn resources

https://scikit-learn.org/stable/modules/impute.html



6.4. Imputation of missing values

For various reasons, many real world datasets contain missing values, often encoded as blanks, NaNs or other placeholders. Such datasets however are incompatible with scikit-learn estimators which assume that all values in an array are numerical, and that all have and hold meaning. A basic strategy to use incomplete datasets is to discard entire rows and/or columns containing missing values. However, this comes at the price of losing data which may be valuable (even though incomplete). A better strategy is to impute the missing values, i.e., to infer them from the known part of the data. See the glossary entry on imputation.

6.4.1. Univariate vs. Multivariate Imputation

One type of imputation algorithm is univariate, which imputes values in the i-th feature dimension using only non-missing values in that feature dimension (e.g. `impute.SimpleImputer`). By contrast, multivariate imputation algorithms use the entire set of available feature dimensions to estimate the missing values (e.g. `impute.IterativeImputer`).

